Baze de date II

CURS 5: TABLE si VARRAY (2); Proceduri, functii si declansatori

Baza de date utilizata. Tabela STUD

MATR	NUME	AN	GRUPA	DATAN	LOC	TUTOR	PUNCTAJ	CODS
1456	GEORGE	4	1141A	12-MAR-82	BUCURESTI		2890	11
1325	VASILE	2	1122A	05-OCT-84	PITESTI	1456	390	11
1645	MARIA	3	1131B	17-JUN-83	PLOIESTI		1400	11
3145	ION	1	2112B	24-JAN-85	PLOIESTI	3251	1670	21
2146	STANCA	4	2141A	15-MAY-82	BUCURESTI		620	21
3251	ALEX	5	2153B	07-NOV-81	BRASOV		1570	21
2215	ELENA	2	2122A	29-AUG-84	BUCURESTI	2146	890	21
4311	ADRIAN	3	2431A	31-JUL-83	BUCURESTI		450	24
3514	FLOREA	5	2452B	03-FEB-81	BRASOV		3230	24
1925	OANA	2	2421A	20-DEC-84	BUCURESTI	4311	760	24
2101	MARIUS	1	2412B	02-SEP-85	PITESTI	3514	310	24
4705	VOICU	2	2421B	19-APR-84	BRASOV	4311	1290	24

Tabelele SPEC si BURSA

CODS	NUME	DOMENIU
11	MATEMATICA	STIINTE EXACTE
21	GEOGRAFIE	UMANIST
24	ISTORIE	UMANIST

TIP	PMIN	PMAX	SUMA
FARA BURSA	0	399	
BURSA SOCIALA	400	899	100
BURSA DE STUDIU	900	1799	150
BURSA DE MERIT	1800	2499	200
BURSA DE EXCEPTIE	2500	9999	300

TABLE si VARRAY - continuare

Manipularea cu SQL - TABLE

- În cazul în care se doreşte actualizarea directă a unei colecţii stocate în baza de date se foloseşte operatorul TABLE (subcerere) care returnează o colecţie de tip TABLE.
- Cererea principală se va aplica colecţiei şi nu liniei din tabelă.
- ***Inserare***: Se adaugă la lista de cursuri 'Limba spaniola':

```
BEGIN
INSERT INTO
   TABLE (SELECT cursuri
   FROM Student
   WHERE id_student = 1234)
VALUES('Limba spaniola');
END;
```

Manipularea cu SQL - TABLE

- ***Actualizare***: Exemplu: se modifică 'Limba spaniola' în 'Limba germana'. Denumirea implicită pentru coloana tabelei returnată de TABLE este COLUMN_VALUE:

```
BEGIN
UPDATE
      TABLE (SELECT cursuri
      FROM Student
      WHERE id_student = 1234)
SET COLUMN_VALUE='Limba germana'
WHERE COLUMN_VALUE='Limba spaniola'
END;
```

Manipularea cu SQL - TABLE

- ***Ştergere.*** Exemplu: Se şterge din listă cursul 'Canto'.

```
BEGIN
DELETE
  TABLE (SELECT cursuri
  FROM Student
  WHERE id_student = 1234)
WHERE COLUMN_VALUE = 'Canto';
END;
```

Manipularea cu SQL - TABLE

- ***Regăsire***. Exemplu: Listarea tuturor cursurilor studentului:

```
SELECT *
FROM
   TABLE (SELECT cursuri
   FROM Student
   WHERE id_student = 1234);
```

- Rezultatul este:

```
COLUMN_VALUE
----------------
Filosofie
Compozitie
Limba italiana
Limba engleza
```

Manipularea cu SQL - TABLE

- ***Regăsire***. Exemplu: Listarea tuturor cursurilor studentului:

```
SELECT *
FROM
     TABLE (SELECT cursuri
     FROM Student
     WHERE id_student = 1234);
```

- Rezultatul este:

```
COLUMN_VALUE
----------------
Filosofie
Compozitie
Limba italiana
Limba engleza
```

- Pentru regăsirea unui curs într-o variabilă putem scrie blocul:
- **DECLARE**

```
v_curs VARCHAR2(30);
BEGIN
SELECT COLUMN_VALUE INTO v_curs
FROM
     TABLE (SELECT cursuri
     FROM Student
     WHERE id_student = 1234)
WHERE COLUMN_VALUE='Limba italiana';
dbms_output.put_line(v_curs);
END;
```

Manipularea cu SQL - TABLE

- Pentru regăsirea unui curs într-o variabilă putem scrie blocul:

```
DECLARE
v_curs VARCHAR2(30);
BEGIN
SELECT COLUMN_VALUE INTO v_curs
FROM
  TABLE (SELECT cursuri
  FROM Student
  WHERE id_student = 1234)
WHERE COLUMN_VALUE='Limba italiana';
dbms_output.put_line(v_curs);
END;
```

Manipularea cu SQL - VARRAY

- În Oracle9i nu este permisă actualizarea directă a unei colecţii de tip VARRAY stocată în baza de date. Regăsirea se face însă folosind ca în exemplul anterior operatorul TABLE (subcerere).
- Pentru operaţii de tip INSERT, UPDATE şi DELETE la nivel de element al unei colecţii de tip VARRAY din baza de date:

1. se regăseşte colecţia într-o variabilă PL/SQL,
2. se actualizează variabila şi
3. se reintroduce în baza de date cu o cerere UPDATE.

Clauza BULK_COLLECT

- În cazul în care o cerere SELECT întoarce mai multe linii fiecare coloană a rezultatului se poate încărca într-o colecţie folosind BULK COLLECT INTO în loc de INTO.
- Iată un exemplu:

```
DECLARE
TYPE TabCod IS TABLE OF stud.matr%TYPE;
TYPE TabNume IS TABLE OF stud.nume%TYPE;
coduri TabCod;
nume TabNume;
BEGIN
SELECT matr, nume BULK COLLECT INTO coduri, nume
FROM stud;
  dbms_output.put_line('Nr.studenti=' ||
       coduri.count);
END;
```

Clauza BULK_COLLECT

- BULK COLLECT se poate folosi şi la FETCH pentru a încărca dintr-un cursor întreg rezultatul în tot atâtea colecţii câte coloane are acesta.
- Exemplu:

```
DECLARE
TYPE TabCod IS TABLE OF stud.matr%TYPE;
TYPE TabNume IS TABLE OF stud.nume%TYPE;
coduri TabCod;
nume TabNume;
CURSOR c IS SELECT matr, nume FROM stud;
BEGIN
OPEN c;
FETCH c BULK COLLECT INTO coduri, nume;
   dbms_output.put_line('Nr.Coduri=' ||
        coduri.count);
END;
```

Clauza BULK_COLLECT

- În acest caz se poate limita numărul maxim de linii încărcate din cursor folosind clauza **LIMIT expresie_intreaga**.
- De exemplu, dacă în programul de mai sus dorim să încărcăm date pentru maxim 5 studenţi instrucţiunea FETCH va fi:

```
FETCH c BULK COLLECT INTO coduri, nume
  LIMIT 5;
```

Clauza BULK_COLLECT

```
FETCH c BULK COLLECT INTO coduri, nume LIMIT 5;
```

- Dacă instrucţiunea de mai sus e plasată într-un ciclu la fiecare pas se vor încărca câte 5 linii - în afară de ultima încărcare care poate aduce mai puţin de 5 linii.
- După ce s-au încărcat toate liniile c%NOTFOUND devine TRUE, ca şi mai înainte.

Excepţii pentru colecţii

- În cazul în care manipularea unei colecţii este defectuoasă se pot ridica o serie de excepţii.
- Exemple:

```
DECLARE
TYPE Lista IS TABLE OF NUMBER;
numere Lista; -- neinitializata deci nula
BEGIN
 numere(1) := 1; -- Exceptie COLLECTION_IS_NULL
 numere := Lista(1,2); -- initializam variabila
 numere(NULL) := 3; -- Exceptie VALUE_ERROR
 numere(0) := 3; -- Exceptie SUBSCRIPT_OUTSIDE_LIMIT
 numere(3) := 3; -- Exceptie SUBSCRIPT_BEYOND_COUNT
 numere.DELETE(1); -- stergem elementul 1
 IF numere(1) = 1 THEN ... -- Exceptie NO_DATA_FOUND
   . . .
 END IF;
END;
```

Instrucţiunea FORALL

- Sintaxa acestei instrucţiuni este:

```
FORALL index IN val_min .. val_max
   cerere_sql;
```

- unde:
  - **index** - o variabilă care nu trebuie declarată şi care are ca domeniu de valabilitate doar cererea SQL. Acest index este indicele care identifică elementele unei colecţii.
  - **val_minima, val_maxima** - arată pentru care indici ai colecţiei se aplică cererea SQL
  - **cerere_sql** - o cerere SQL în care apare o colecţie indexată după indicele **index**.

Instrucţiunea FORALL

- Instrucţiunea FORALL nu este o instrucţiune de ciclare, în sensul că nu se face un dialog între PL/SQL şi serverul Oracle pentru fiecare valoare a indexului.
- Ea se foloseşte doar ca o indicare a plajei de valori implicată în cererea SQL.
- Exemplu:

```
DECLARE
TYPE Lista IS VARRAY(10) OF NUMBER;
spec Lista := Lista(21,22,23,24,25);
BEGIN
 FORALL i IN 2..4
  UPDATE stud
  SET punctaj = punctaj * 1.10
WHERE cods = spec(i);
END;
```

Instrucţiunea FORALL

- Efectul în acest caz este acelaşi cu al execuţiei cererii:

```
UPDATE stud
SET punctaj = punctaj * 1.10
WHERE cods in (22,23,24);
```

FORALL - %BULK_ROWCOUNT

- În cazul folosirii lui FORALL programul poate afla câte linii au fost afectate la fiecare pas folosindu-se atributul %BULK_ROWCOUNT a cursorului implicit SQL.
- Acesta este compus din mai multe valori, câte una pentru fiecare valoare a indicelui.
- De exemplu, pentru programul anterior plaja era 2..4.
- Exemplu:

```
DECLARE
TYPE Lista IS VARRAY(10) OF NUMBER;
spec Lista := Lista(21,22,23,24,25);
BEGIN
FORALL i IN 2..4
  UPDATE stud
  SET punctaj = punctaj * 1.10
WHERE cods = spec(i);
dbms_output.put_line('Pasul 3, afectate '||
       SQL%BULK_ROWCOUNT(4)||' linii');
END;
```

FORALL - Exceptii

- Toate elementele colecţiei referite de index trebuie să existe.
- De exemplu, programul următor ridică o excepţie:

```
DECLARE
TYPE  Lista IS TABLE OF NUMBER;
spec Lista := Lista(21,22,23,24,25,26,27,28,29);
BEGIN
spec.Delete(5);
   -- stergem elementul cu indice 5
FORALL i IN 2..8
   -- plaja cuprinde elementul sters
  UPDATE stud SET punctaj = 0 WHERE cods =
  spec(i);
EXCEPTION
  WHEN OTHERS THEN COMMIT;
END;
```

FORALL - Exceptii

- Dacă o excepţie apărută în FORALL nu este tratată toate modificările făcute sunt anulate (ROLLBACK inclusiv pentru execuţiile i=2, 3, 4).
- Dacă însă, ca în exemplul de mai sus, erorile sunt tratate, actualizările vor fi comise pentru valorile 2, 3 şi 4 ale lui i.
- La valoarea 5 se ridică excepţia şi cererea UPDATE nu se mai execută pentru valorile următoare (6, 7, etc.).
- De asemenea se face implicit un ROLLBACK pentru cererea care a ridicat excepţia (UPDATE pentru valoare 5 a lui i).

Clauza SAVE EXCEPTIONS

- Se poate folosi clauza SAVE EXCEPTIONS care specifică faptul că orice excepţie ridicată este salvată în atributul %BULK_EXCEPTIONS ataşat cursorului SQL
- Acesta conţine o colecţie de înregistrări cu două câmpuri fiecare, ERROR_INDEX şi ERROR_CODE iar execuţia merge mai departe.
- Saltul în zona de excepţii se face abia după terminarea iteraţiilor.

Clauza SAVE EXCEPTIONS

- In zona de exceptii putem folosi:
  - **SQL%BULK_EXCEPTIONS(i).ERROR_INDEX** - indexul FORALL la care a apărut excepţia,
  - **SQL%BULK_EXCEPTIONS(i).ERROR_CODE** - codul erorii respective,
  - **SQL%BULK_EXCEPTIONS.COUNT** - câte excepţii au apărut.
- Observaţie: indicele **i** de mai sus poate lua valori cuprinse între 1 şi valoarea lui SQL%BULK_EXCEPTIONS.COUNT. Acest fapt poate fi folosit pentru a raporta excepţiile apărute folosind un ciclu FOR.

Clauza SAVE EXCEPTIONS

- În acest caz sintaxa unei instrucţiuni FORALL este:

```
FORALL index IN val_min .. val_max
  SAVE EXCEPTIONS
   {cerere_insert | cerere_update |
  cerere_delete}
```

Exemplu

- În blocul următor apare de trei ori excepţia de împarţire la zero:

```
DECLARE
  TYPE Lista IS TABLE OF NUMBER;
  numere Lista := Lista(10,0,11,12,30,0,20,199,2,0,9,1);
  erori NUMBER;
BEGIN
  FORALL i IN numere.FIRST..numere.LAST SAVE EXCEPTIONS
    DELETE FROM stud WHERE punctaj > 500000/numere(i);
EXCEPTION
  WHEN OTHERS THEN
  erori := SQL%BULK_EXCEPTIONS.COUNT;
  dbms_output.put_line('Nr.de erori= ' || erori);
  FOR i IN 1..erori LOOP
    dbms_output.put_line('Eroarea ' || i ||
       ' aparuta la iteratia ' ||
       SQL%BULK_EXCEPTIONS(i).ERROR_INDEX);
    dbms_output.put_line('Eroarea Oracle este ' ||
       SQLERRM(-SQL%BULK_EXCEPTIONS(i).ERROR_CODE));
  END LOOP;
END;
```

Exemplu

- Rezultatul execuţiei blocului este:

```
Nr.de erori= 3
Eroarea 1 aparuta la iteratia 2
Eroarea Oracle este ORA-01476: divisor is
  equal to zero
Eroarea 2 aparuta la iteratia 6
Eroarea Oracle este ORA-01476: divisor is
  equal to zero
Eroarea 3 aparuta la iteratia 10
Eroarea Oracle este ORA-01476: divisor is
  equal to zero
```

Proceduri, functii, pachete si declansatori

Ce sunt acestea?

- Procedurile şi funcţiile sunt denumite şi subprograme.
- Ele sunt blocuri PL/SQL având un nume, o listă de parametri şi, în cazul funcţiilor, o valoare returnată.
- Mai multe proceduri şi/sau funcţii pot fi 'împachetate' sub forma unui 'pachet' (package).

Declararea unei proceduri

- Sintaxa declarării unei proceduri este:

Doar pentru proceduri stocate

```
[CREATE [OR REPLACE]]
PROCEDURE nume_procedura[(parametru[,
  parametru]...)]
[AUTHID {DEFINER | CURRENT_USER}] {IS | AS}
[PRAGMA AUTONOMOUS_TRANSACTION;]
[declaratii locale]
BEGIN
instructiuni executabile sau NULL;
[EXCEPTION
tratare erori]
END [nume_procedura];
```

Declararea unei proceduri

- **[CREATE [OR REPLACE]]** - arată că procedura se defineşte ca obiect al bazei de date şi va fi stocată în aceasta. Altfel ea este parte a unui bloc PL/SQL.
- **[AUTHID {DEFINER | CURRENT_USER}]** - specifică dacă o procedură stocată se execută cu drepturile celui care a creat-o (valoare implicită) sau ale utilizatorului curent.
- **[PRAGMA AUTONOMOUS_TRANSACTION;]** - dacă se specifică această caracteristică, execuţia procedurii suspendă tranzacţia curentă care se reia după terminarea execuţiei acesteia.

Parametri formali

- Fiecare parametru formal se defineşte astfel:

```
nume_parametru [IN | OUT [NOCOPY] |
  IN OUT [NOCOPY]]
  numetip [{:= | DEFAULT} expresie]
```

- unde:

A. **IN**, **OUT** şi **IN OUT** arată că este vorba de un parametru de intrare, de ieşire sau bidirecţional. În cazul în care nu se specifică, parametrul este implicit transmis ca IN. În corpul unui subprogram nu se poate asigna o valoare unui parametru transmis cu IN.

Parametri formali

B. **NOCOPY**. În mod implicit parametrii de tip IN sunt transmişi prin referinţă iar cei OUT şi IN OUT prin valoare.

- În cazul unor parametri voluminoşi transmiterea prin valoare e consumatoare de resurse.
- Se poate specifica în acest caz (OUT, IN OUT) transmiterea prin referinţă folosind NOCOPY.

Parametri formali

C. **numetip** este un nume de tip fără specificarea dimensiunii (deci VARCHAR2 de exemplu şi nu VARCHAR2(10).

- Se poate totuşi asocia o dimensiune maximă prin definirea unui subtip care să corespundă dorinţelor noastre.

Parametri formali

- Exemplu:

```
DECLARE
SUBTYPE Char25 IS CHAR(25);
PROCEDURE cauta (persoana
  Char25) IS ...
```

Plasare

- Când o procedură este parte a unui alt bloc ea va fi plasată în zona DECLARE a acestuia şi poate fi apelată în zona executabilă cu parametrii actuali.
- Exemplu: procedura ***alt_tutor*** schimbă valoarea de pe coloana TUTOR a unui student.
- Ea nu execută operaţia în cazul în care studentul nu are un tutor sau când codul tutorului este identic cu al studentului:

```
DECLARE
 PROCEDURE alt_tutor (cod_stud INTEGER, nou_tutor
  INTEGER)
 IS
   tutor_actual number;
   fara_tutor EXCEPTION; este_el EXCEPTION;
 BEGIN
   SELECT tutor INTO tutor_actual FROM stud1
   WHERE matr = cod_stud;
```

Plasare

```
IF nou_tutor=cod_stud THEN
    RAISE este_el;
 ELSIF tutor_actual IS NULL THEN
    RAISE fara_tutor;
 ELSE
    UPDATE stud1 SET tutor = nou_tutor
    WHERE matr = cod_stud;
    dbms_output.put_line('Actualizat
'||cod_stud);
 END IF;
```

Plasare

```
EXCEPTION
   WHEN NO_DATA_FOUND THEN
     dbms_output.put_line('Nu exista studentul');
   WHEN fara_tutor THEN
     dbms_output.put_line('Nu are tutor');
   WHEN este_el THEN
     dbms_output.put_line('Nu isi poate fi tutor');
  END alt_tutor;
BEGIN  -- blocul principal
  alt_tutor(1325, 1645);
  alt_tutor(1456, 1645);
  alt_tutor(1645, 1645);
END;
Rezultatul va fi:
Actualizat 1325
Nu are tutor
Nu isi poate fi tutor
```

Declararea unei funcţii

- Sintaxa simplificată a declaraţiei unei funcţii este următoarea:

Doar pentru functii stocate

```
[CREATE [OR REPLACE ] ]
FUNCTION nume_functie [ ( parametru [ , parametru
  ]... )]
 RETURN tip_date_returnate
 [ AUTHID { DEFINER | CURRENT_USER } ]
{IS | AS}
[ PRAGMA AUTONOMOUS_TRANSACTION; ]
[declaratii locale]
BEGIN
instructiuni executabile sau NULL;
[EXCEPTION
tratare erori]
END [nume_functie];
```

Declararea unei funcţii

- Elementele care apar şi la proceduri au aceeaşi semnificaţie.
- În plus apare tipul valorii returnate care, ca şi parametrii formali, nu poate fi constrâns prin specificarea unei dimensiuni maxime.
- În zona executabilă trebuie să apară instrucţiunea **RETURN expresie** care specifică valoarea returnată de funcţie.

Exemplu

- Programul de schimbare tutor rescris prin transformarea procedurii într-o funcţie care întoarce un şir de caractere:

```
DECLARE
 FUNCTION alt_tutor (cod_stud INTEGER,
  nou_tutor INTEGER)
 RETURN VARCHAR2 IS
    tutor_actual number;
    fara_tutor EXCEPTION;
    este_el EXCEPTION;
 BEGIN
    SELECT tutor INTO tutor_actual FROM stud1
    WHERE matr = cod_stud;
```

Exemplu

```
  IF nou_tutor=cod_stud THEN
    RAISE este_el;
  ELSIF tutor_actual IS NULL THEN
    RAISE fara_tutor;
  ELSE
    UPDATE stud1 SET tutor = nou_tutor
    WHERE matr = cod_stud;
    RETURN 'Actualizat '||cod_stud;
  END IF;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RETURN 'Nu exista studentul';
  WHEN fara_tutor THEN
    RETURN 'Nu are tutor';
  WHEN este_el THEN
    RETURN 'Nu isi poate fi tutor';
 END alt_tutor;
```

Exemplu

```
BEGIN  -- blocul principal
  dbms_output.put_line(alt_tutor(1325, 1645));
  dbms_output.put_line(alt_tutor(1456, 1645));
  dbms_output.put_line(alt_tutor(1645, 1645));
END;
```

- După cum se poate observa RETURN poate fi prezent şi în zona de tratare a excepţiilor.

Apelarea unei funcţii în SQL

- Pentru ca o funcţie stocată să fie apelabilă în cereri SQL trebuie să respecte următoarele restricţii:

1. Când este apelată într-o cerere SELECT funcţia nu trebuie să modifice nimic în tabelele bazei de date.
2. Când este apelată în INSERT, UPDATE sau DELETE funcţia nu poate regăsi sau modifica tabela actualizată de cererea SQL.

Apelarea unei funcţii în SQL

3. Când este apelată din SELECT, INSERT, UPDATE sau DELETE funcţia nu poate executa:
   - cereri de control pentru tranzacţii (ex.: COMMIT),
   - cereri de control pentru sesiunea de lucru (ex.: SET ROLE) sau
   - cereri de control sistem (ca ALTER SYSTEM).
   - De asemenea nu poate executa cereri care se comit automat (ca de exemplu CREATE).

Parametrii unui subprogram: IN

Modul IN este modul implicit de transmitere.

- Parametrul este un parametru de intrare.
- Parametrul este transmis prin referinţă, deci se transmite o adresă a originalului.
- Parametrul acţionează ca o constantă: în corpul subprogramului acesta nu poate fi modificat (de exemplu prin atribuire sau SELECT INTO), o astfel de operaţie ducând la erori de compilare.
- La apelarea subprogramului parametrul actual poate fi o constantă, un literal, o variabilă iniţializată sau o expresie.

Parametrii unui subprogram: IN

- Spre deosebire de celelalte două moduri pentru aceşti parametri se pot defini valori implicite (DEFAULT).
- Specificarea unei valori implicite se face ca în exemplul următor:

```
FUNCTION marire_bursa(
cod_stud INTEGER,
suma REAL DEFAULT 100) RETURN VARCHAR2
  IS . . .
```

- În acest caz funcţia se poate chema cu 2 parametri actuali dar şi cu unul singur, în al doilea caz al doilea având valoarea implicită 100:

```
marire_bursa(1645, 200)
marire_bursa(1645)
```

Parametrii unui subprogram: IN

- În cazul următor însă:

```
FUNCTION marire_bursa(
suma REAL DEFAULT 100,
cod_stud INTEGER) RETURN VARCHAR2
   IS . . .
```

- Apelurile urmatoare semnalează eroare şi nu actualizează bursa studentului cu matricola 1645.

```
marire_bursa(1645)
marire_bursa(, 1645)
```

- Rezolvarea acestor tipuri de situaţii este prezentată la sfârşitul acestui subcapitol.

Parametrii unui subprogram: OUT

- Parametrul este un parametru de ieşire.
- Parametrul este transmis prin valoare, deci se transmite o copie a originalului.
- Parametrul poate fi modificat în corpul subprogramului.
- La apelarea subprogramului parametrul actual trebuie să fie o variabilă.
- La apelare acesta poate avea o valoare care se pierde însă la apel (în afara cazului când se specifică NOCOPY).

Parametrii unui subprogram: OUT

- În momentul apelului parametrul este iniţializat cu NULL. Din acest motiv tipul lui nu poate fi unul constrâns de NOT NULL (ex.: POSITIVEN, NATURALN)
- Parametrului nu i se poate asocia o valoare implicită (DEFAULT).
- În corpul subprogramului trebuie să i se asocieze o valoare (altfel rămâne cu valoarea NULL).
- La ieşirea din subprogram PL/SQL asignează valoarea respectivă (inclusiv NULL) parametrului actual.
- Dacă subprogramul iese cu o excepţie netratată asignarea de mai sus nu este făcută!

Parametrii unui subprogram: INOUT

- Parametrul este un parametru de intrare şi de ieşire.
- Parametrul este transmis prin valoare, deci se transmite o copie a originalului.
- Parametrul poate fi modificat în corpul subprogramului.
- La apelarea subprogramului parametrul actual trebuie să fie o variabilă iniţializată care se poate folosi în corpul subprogramului.

Parametrii unui subprogram: INOUT

- Parametrului nu i se poate asocia o valoare implicită (DEFAULT).
- În corpul subprogramului parametrului i se poate asocia o valoare (altfel aceasta rămâne valoarea de la apel).
- La ieşirea din subprogram PL/SQL asignează valoarea respectivă (inclusiv NULL) parametrului actual.
- Daca subprogramul iese cu o excepţie netratată asignarea de mai sus nu este făcută!

Opţiunea NOCOPY

- Se aplică pentru parametrii OUT sau IN OUT specificând că transmiterea se face prin referinţă nu prin valoare.
- În unele cazuri ea este ignorată de către sistem, deci transmiterea se face tot prin valoare.
- Protocolul RPC nu permite decât transmiterea prin valoare. În cazul mutării unei proceduri având parametri NOCOPY pe alt host automat la folosirea ei aceştia vor fi transmişi prin valoare.

Opţiunea NOCOPY

- În cazul ieşirii din subprogram cu excepţie netratată nu se poate şti exact starea parametrului actual:
  - dacă s-a transmis totuşi prin referinţă modificările făcute până la apariţia excepţiei se reflectă în valoarea parametrului actual
  - dacă transmiterea s-a făcut prin valoare PL/SQL nu copiază valorile în parametrul actual.

Specificarea valorilor parametrilor

- Există posibilitatea ca la apelul unuj subprogram parametrii actuali să fie în altă ordine decât cei formali. În acest caz se specifică pentru parametrii formali care sunt parametrii actuali asociaţi:
- Exemplu. Funcţia:

```
FUNCTION marire_bursa(
suma REAL DEFAULT 100,
cod_stud INTEGER) RETURN VARCHAR2
  IS . . . .
```

- poate fi apelată în următoarele moduri:

```
marire_bursa(200, 1645); -- pozitional
marire_bursa(suma => 200, cod_stud => 1645); -- cu nume
marire_bursa(cod_stud => 1645, suma => 200); -- cu nume
marire_bursa(cod_stud => 1645);  -- cu nume; suma = 100
marire_bursa(200, cod_stud => 1645); -- mixt
```

DROP

- Ştergerea unei proceduri sau funcţii se face cu DROP:
- Sintaxa:

```
DROP PROCEDURE nume_procedura;
DROP FUNCTION nume_functie;
```

- Exemple:

```
DROP FUNCTION marire_bursa;
DROP PROCEDURE alt_tutor;
```

Pachete

- Mai multe proceduri şi/sau funcţii pot forma un pachet (package). Declararea unui pachet se face în două etape:

1. Declararea specificaţiei pachetului.
2. Declararea corpului său conţinând implementarea procedurilor şi funcţiilor componente.

Specificatie

- Specificaţia unui pachet conţine declaraţiile publice aferente acestuia, vizibile pentru aplicaţiile care îl folosesc.
- Sintaxa simplificată a declarării specificaţiei este următoarea:

```
CREATE [OR REPLACE] PACKAGE nume_pachet
[AUTHID {CURRENT_USER | DEFINER}]
{IS | AS}
[definitii tipuri pt. colectii si inregistrari;
 subtipuri]
[declarare colectii, constante, exceptii,
  obiecte, inregistrari, variabile si cursori]
[declaratii functii si proceduri]
END [nume_pachet];
```

Implementare (corp)

- Specificaţia unui pachet conţine declaraţiile publice aferente acestuia, vizibile pentru aplicaţiile care îl folosesc.
- Sintaxa simplificată a declarării specificaţiei este următoarea:

```
CREATE [OR REPLACE] PACKAGE BODY nume_pachet
[AUTHID {CURRENT_USER | DEFINER}]
{IS | AS}
[definitii tipuri pentru colectii si
  inregistrari; subtipuri]
[declarare colectii, constante, exceptii,
  obiecte, inregistrari, variabile si
  cursori]
[declaratii functii si proceduri]
END [nume_pachet];
```

Exemplu

- Pachetul STD conţine funcţiile NUME şi PUNCTE care returnează numele şi punctajul unui student. Funcţiile au ca parametru numărul matricol al acestuia:

```
CREATE OR REPLACE PACKAGE STD AS
   FUNCTION NUME (
      PMATR NUMBER) RETURN VARCHAR2;
   FUNCTION PUNCTE (
      PMATR NUMBER) RETURN NUMBER;
END STD;
```

Exemplu

```
CREATE OR REPLACE PACKAGE BODY STD
  AS
   FUNCTION NUME (
      PMATR NUMBER) RETURN VARCHAR2 IS
   V_NUME VARCHAR2(10);
   BEGIN
      SELECT NUME INTO V_NUME
      FROM STUD
      WHERE MATR=PMATR;
      RETURN V_NUME;
   END NUME;
```

Exemplu

```
FUNCTION PUNCTE (
     PMATR NUMBER) RETURN NUMBER IS
   V_PUNCTE NUMBER(4);
   BEGIN
     SELECT PUNCTAJ INTO V_PUNCTE
     FROM STUD
     WHERE MATR=PMATR;
     RETURN V_PUNCTE;
   END PUNCTE;
END STD;
```

Apelare

- Folosirea subprogramelor care fac parte dintr-un pachet se face prin construcţia:

```
Pachet.subprogram([param_actuali])
```

- Exemplu: Cererea următoare returnează numele studentului, numele tutorului său şi punctajul acestuia:

```
SELECT NUME, STD.NUME(TUTOR) TUTOR,
       STD.PUNCTE(TUTOR) PUNCTE
FROM STUD;
```

- Rezultatul conţine valori nule pentru studenţii care nu au un tutor:

Apelare

NUME	TUTOR	PUNCTE
GEORGE		
VASILE	GEORGE	2890
MARIA		
ION	ALEX	1570
STANCA		
ALEX		
ELENA	STANCA	620
ADRIAN		
FLOREA		
OANA	ADRIAN	450
MARIUS	FLOREA	3230
VOICU	ADRIAN	450

Stergere pachet

- Ştergerea unui pachet se face folosind cereri DROP:

```
DROP PACKAGE BODY nume_pachet;
DROP PACKAGE nume_pachet;
```

- Exemplu:

```
DROP PACKAGE BODY STD;
DROP PACKAGE STD;
```

Declanşatori (triggers)

- Execuţia unei proceduri, funcţii sau bloc PL/SQL se poate declanşa automat la apariţia unui eveniment dacă există un declanşator (în engleză ***trigger***) asociat cu evenimentul respectiv.
- De exemplu orice modificare a punctajului unui student (folosind UPDATE) poate declanşa automat inserarea unei linii într-o tabelă de istoric unde este salvată vechea valoare a punctajului.
- Sintaxa simplificată a definirii unui declanşator este următoarea:

Declanşatori (triggers)

```
CREATE [OR REPLACE] TRIGGER nume_trigger
  {BEFORE | AFTER | INSTEAD OF}
  {DELETE | INSERT | UPDATE [OF col1 [, col2]
  ...]}
[OR
  {DELETE | INSERT | UPDATE [OF col3 [, col4]
  ...]}
] ...
ON nume_tabela
  [ [REFERENCING { OLD [AS] old [NEW [AS] new]
                 | NEW [AS] new [OLD [AS] old] }
  ]
[ FOR EACH ROW ]
  [WHEN (conditie)] ]
    bloc_PL_SQL
```

Descriere elemente

- **CREATE OR REPLACE** - Creaza sau inlocuieşte vechea definiţie a declanşatorului dacă acesta există deja. În cazul în care OR REPLACE lipseşte şi există deja un declanşator cu numele respectiv Oracle semnalează o eroare.
- **nume_declansator** - numele declanşatorului (identificator).
- **BEFORE** - declanşarea se face înaintea evenimentului vizat.
- **AFTER** - declanşarea se face după evenimentul vizat.
- **INSTEAD OF** - acţiunea declanşatorului înlocuieşte evenimentul vizat.
- **DELETE** - declanşarea se face la apariţia unei comenzi DELETE.
- **INSERT** - declanşarea se face la apariţia unei comenzi INSERT.
- **UPDATE...OF** - declanşarea se face la apariţia unei comenzi UPDATE care actualizează una dintre coloanele specificate (dacă sunt specificate).

Descriere elemente

- **ON** - specifică tabela careia îi este asociat declanşatorul.
- **REFERENCING** - specifică nume alternative pentru valorile vechi şi noi ale liniei. Implicit acestea sunt referite cu prefixele :old si :new.
- **FOR EACH ROW** - arată că este vorba de un declanşator la nivel de linie: acţiunea asociată se execută pentru fiecare linie afectată de evenimentul care îl declanşează.
- În lipsa acestei clauze va fi definit un declanşator la nivel de cerere (se execută o singură dată indiferent de numărul de linii afectate de evenimentul vizat).

Descriere elemente

- **WHEN** - specifică o condiţie suplimentară. Se utilizează în declanşatorii la nivel de linie şi arată că acţiunea asociată se va executa doar pentru liniile afectate care îndeplinesc condiţia WHEN.
- **bloc_PL_SQL** - acţiunea executată la declanşare.
- Dacă în zona de declaraţii blocul PL/SQL conţine:

  `PRAGMA AUTONOMOUS_TRANSACTION`

  atunci operaţiile de COMMIT şi ROLLBACK din interiorul declanşatorului sunt independente (pot fi de exemplu comise şi în cazul în care modificările care au declanşat acţiunea sunt anulate cu ROLLBACK).

Descriere elemente

- Declanşatorii se pot asocia cu:
- Cereri DML: DELETE, INSERT sau UPDATE.
- Cereri DDL: CREATE, ALTER sau DROP.
- Operatii ale bazei de date: SERVERERROR, LOGON, LOGOFF, STARTUP sau SHUTDOWN)

Exemplu

- În exemplul următor prezentăm doi declanşatori

1. unul la nivel de linie care salvează într-o tabelă ISTORIC valorile punctajelor studenţilor atunci când coloana respectivă este actualizată:

```
CREATE OR REPLACE TRIGGER scrie_istorie
  AFTER UPDATE OF punctaj ON stud
  FOR EACH ROW
BEGIN
    INSERT INTO ISTORIC
      VALUES (:old.MATR, :old.punctaj,
  :new.punctaj);
END;
```

Exemplu

- În exemplul următor prezentăm doi declanşatori:

2. unul la nivel de cerere care înregistrează cine a modificat tabela SPEC şi când (dar nu şi cum):

```
CREATE OR REPLACE TRIGGER cine_modifica
   BEFORE INSERT OR DELETE OR UPDATE ON
  spec
   BEGIN
      INSERT INTO MODIF VALUES (USER,
  SYSDATE);
   END;
```

Exemplu

- Efectul unei actualizări a tabelei STUD asupra conţinutului tabelei ISTORIC este următorul:

```
UPDATE STUD
SET PUNCTAJ = PUNCTAJ + 100
WHERE CODS = 11;

SELECT * FROM ISTORIC;

 MATR PVECHI  PNOU
----- ------ -----
 1456   2890  2990
 1325    390   490
 1645   1400  1500
```

Alt exemplu

- Se poate folosi un declansator (trigger) pentru generarea unei chei surogat folosind o secventa:

```
CREATE SEQUENCE SECV;
--
CREATE TABLE TEST
  (PK NUMBER PRIMARY KEY,
  NUME VARCHAR2(40) NOT NULL,
  MEDIE NUMBER);
```

Alt exemplu

```
CREATE OR REPLACE TRIGGER NOU_PK
  BEFORE INSERT ON TEST
     FOR EACH ROW
  BEGIN
     IF :NEW.EMPNO IS NULL THEN
       :NEW.EMPNO := SECV.NEXTVAL;
     END IF;
END;
--
```

Alt exemplu

```
INSERT INTO TEST (NUME, MEDIE) VALUES
  ('ION', 10.00);
INSERT INTO TEST (NUME, MEDIE) VALUES
  ('DAN', 9.90);
INSERT INTO TEST VALUES (7, 'MARIA',
  10.00);
COMMIT;
```

- Rezultat: inserarea a 3 linii cu cheile 1, 2 si 7 (de retinut ca secventa SECV va genera mai tarziu valoarea 7!).

:new si :old

- :new si :old se pot folosi astfel:

	:NEW	:OLD
INSERT	√	N/A
UPDATE	√	√
DELETE	N/A	√

Stergere

- Ştergerea unui declanşator se face cu DROP TRIGGER. Exemplu:

```
DROP TRIGGER scrie_istorie;
```

- Observaţie: în cazul în care la crearea unui obiect de tip procedură, funcţie, pachet sau declanşator se semnalează erori acestea pot fi vizualizate cu comanda:

```
SHOW ERRORS;
```

Sfârşitul cursului 5 si al capitolului de PL/SQL